



8b-vm-os

Bu PDF'te ise İşletim Sisteminin bir sihirbaz gibi davranarak, aslında RAM'de olmayan sayfaları "sanki oradaymış gibi" programa nasıl yutturduğunu öğreneceğiz.

Kısmi Paging

Tembel Yükleyici (Lazy Swapper/Pager)

Bir programı açtığınızda OS programın tamamını ram'e yüklemeyiz. Çünkü 80/100 kuralı vardır. Bir programın kodunun %20'si, çalışma zamanının %80'ini oluşturur. Hata mesajları, nadir kullanılan menüler, devasa veri yapıları belki de o gün hiç kullanılmayacaktır.

İşte işletim sistemi **İsteğe Bağlı Sayfalama (Demand Paging)** denilen bir "Tembel Yükleyici" kullanır. Sadece ve sadece programın o an "Bana bu lazım" dediği sayfaları RAM'e getirir. Geri kalanı diskte uyumaya devam eder.

Sayfa Hatası (Page Fault) Nedir ve 6 Adımda Nasıl Çözülür?

Sanal hafızada her sayfanın yanında bir **Geçerli/Geçersiz (Valid/Invalid - v/i)** tiki vardır.

- **v (Valid):** Bu sayfa şu an fiziksel RAM'de (Kulüpte).
- **i (Invalid):** Bu sayfa diskte (Evde).

Peki ya işlemci gidip yanında "i" (Invalid) yazan bir adresi okumaya kalkarsa ne olur? İşlemci anında donanımsal bir alarm çalar. Buna **Sayfa Hatası (Page Fault)** diyoruz. Bu bir çökme hatası (error) değildir, işletim sistemine "Acil yardıma gel" çağrısıdır (Trap). İşletim sistemi bu krizi şu 6 adımda çözer:

1. İşlemci adrese bakar, "i" olduğunu görür ve işletim sistemine alarm yollar.
2. İşletim sistemi araya girer (trap) ve programı durdurur.
3. Adresin gerçekten o programa mı ait olduğunu yoksa yasadışı bir erişim mi olduğunu kontrol eder. Adres yasalysa, sayfanın diskteki (Hard Disk/SSD) yerini bulur.
4. Diske gidip o sayfayı kopyalar ve RAM'de boş bir çerçeveye (Free Frame) getirir.
5. Programın "Adres Defterini" (Sayfa Tablosunu) günceller, "i" harfini "v" (Valid) yapar.
6. Dondurduğu o işlemi sanki hiçbir şey olmamış gibi **en baştan tekrar başlatır (Instruction Restart)**.

Etkin Erişim Süresi (Effective Access Time - EAT) Formülü

Bu formül, sayfa hatalarının sistemimizi ne kadar korkunç yavaşlattığını kanıtlar:

$$EAT = (1 - p) \times (\text{RAM'den Okuma Süresi}) + p \times (\text{Sayfa Hatası Maliyeti})$$

- **p (Olasılık):** Aradığımız sayfanın RAM'de olmama (Page Fault) ihtimalidir.
- **Mantık:** Eğer sayfa RAM'deyse $(1 - p)$ süper hızlıca (örn: 200 nanosaniye) alırız. Ama sayfa RAM'de yoksa (p) gidip yavaş diskten getirmek zorundayızdır (örn: 8.000.000 nanosaniye - yani 8 milisaniye).
- **Sonuç:** PDF'teki hesaba göre, 1000 işlemten sadece 1 tanesi bile RAM'de yoksa $(p = 0.001)$, bilgisayarının hızı bir anda **40 kat** yavaşlar! İşte bu yüzden "Sayfa Hatası" oranını minimumda tutmak işletim sisteminin en büyük varoluş amacıdır.

Copy-On-Write (COW)

Bir işlem (process) kendini kopyalayıp yeni bir çocuk işlem yarattığında (`fork()`), normalde işletim sisteminin tüm RAM'i (örneğin 2 GB'lık veriyi) kelimesi kelimesine kopyalaması gerekir. Bu korkunç bir israftır.

Bunun yerine İşletim Sistemi **Copy-On-Write (Yazarken Kopyala)** hilesini kullanır:

- Çocuk yaratıldığında hiçbir şey kopyalanmaz. Çocuğun adres defteri, tıpkı ebeveyni gibi **birebir aynı fiziksel RAM sayfalarını** gösterir. (Okuma yaptıkları sürece aynı kitaba bakarlar).
- Ne zaman ki çocuk veya ebeveyn ortak sayfalardan birini **değiştirmeye (yazmaya/write)** kalkar, işte sadece o saniye işletim sistemi araya girer, *"Hop, artık ortaklık bozuldu"* der ve sadece o tek sayfanın (4 KB) fiziksel bir kopyasını çıkarıp değiştiren kişiye verir.

Böylece sadece değişen kısımlar RAM'de fazladan yer kaplar. Muazzam bir hız ve hafıza tasarrufudur!

Sayfa Değiştirme Algoritmaları ve Belady Anomalisi

RAM ağzına kadar dolduğunda ve yepyeni bir sayfa içeri girmek istediğinde ne olacak?

İşletim sisteminin çok acımasız bir karar vermesi gerekir: **"İçeridekilerden birini kurban (victim) seçip diske geri şutlamalıyım ki yeni gelene yer açılsın."** İşte bu bölüme "Sayfa Değiştirme" (Page Replacement) diyoruz.

"Kirli" ve "Temiz" Sayfalar (Dirty Bit Mantığı)

Kimi dışarı atacağımızı seçmeden önce bilmemiz gereken hayati bir hız kuralı vardır. Seçtiğimiz kurbanın iki durumu olabilir:

- **Temiz (Clean):** Kurban RAM'e girdiğinden beri içindeki veriyi hiç değiştirmemiştir (örneğin sadece okunmuş bir kod sayfası). Bunu evden atarken diske geri yazmamıza gerek yoktur, çünkü aynısı zaten diskte var. Direkt üstüne yazar geçeriz, inanılmaz hızlıdır.
- **Kirli (Dirty):** Kurban RAM'de kalırken içindeki bir değişkeni/veriyi değiştirmiştir. Bunu evden atarken mecburen o değişiklikleri diske kaydetmek (yazmak) zorundayızdır. Bu çok yavaş bir işlemdir.

Bu yüzden işletim sistemleri kurban seçerken her zaman "Temiz" olanları atmaya meillidir.

FIFO (İlk Gelen İlk Çıkar) ve "Belady Anomalisi"

En basit mantık şudur: "Kardeşim sen RAM'e ilk giren kişisin, en uzun süredir buradasın, o zaman ilk sen çıkacaksın." (FIFO - First In First Out).

Ama bu mantık korkunç bir hataya yol açar. Belki de o ilk giren sayfa, programın ana döngüsüdür ve saniyede binlerce kez kullanılıyordur! Sen onu atarsan, saniye dolmadan geri çağırmak zorunda kalırsın.

Belady Anomalisi

- Normalde matematikte bir programa daha fazla RAM (daha fazla boş çerçeve) verirsen, sayfa hatası (page fault) ihtimalinin *azalmasını* beklersin, değil mi?
- Belady adında bir bilim insanı kanıtlamıştır ki; eğer FIFO algoritmasını kullanıyorsan, RAM'i büyütme (örneğin 3 kutudan 4 kutuya çıkarmak) bazı durumlarda hata sayısını **azaltmak yerine artırır!** Sisteme daha çok kaynak verip onu daha da bozabildiğin bu tuhaf duruma Belady Anomalisi diyoruz. FIFO bu yüzden çöpe atılmıştır.

OPT ve LRU Algoritmaları

OPT (Kahin Algoritması)

"Gelecekte en uzun süre kullanılmayacak olan sayfayı atalım." Mükemmeldir ama imkansızdır, çünkü işletim sistemi geleceği göremez. Sadece diğer algoritmaların ne kadar iyi olduğunu ölçmek için bir "zirve noktası" (benchmark) olarak kullanılır.

LRU (En Az Yakın Zamanda Kullanılan)

Madem geleceği göremiyoruz, geçmişe bakalım! "Geçmişte en uzun süredir yüzüne bakılmayan (kullanılmayan) sayfayı atalım." LRU harikadır, Belady anomalisine yakalanmaz ama çok büyük bir derdi vardır: **Çok pahalıdır!** Her sayfanın ne zaman kullanıldığının saniye saniye kaydını (Zaman damgası veya Stack ile) tutmak, işlemciyi felç eder.

Saat Algoritması (İkinci Şans - Second Chance)

İşletim sistemleri (Linux vb.) LRU'nun bu pahalı maliyetinden kaçmak için **Saat (Clock)** adında harika bir hile bulmuşlardır.

- RAM'deki bütün sayfaları yuvarlak bir masa etrafına diziyoruz. Ortada da dönen bir saat yelkovanı (işaretçi) var.
- Her sayfanın yakasına bir **"Kullanıldı Tiki" (Reference Bit)** takıyoruz. İşlemci o sayfaya her dokunduğunda bu tik "1" oluyor.
- Yeni bir kurban gerektiğinde yelkovan dönmeye başlıyor ve bir sayfada duruyor. Şuna bakıyor: *"Yakındaki tik 1 mi, 0 mı?"*
- Eğer **1 ise**, yelkovan der ki: *"Demek ki yakın zamanda kullanılmışsın. Seni atmıyorum ama tikini 0 yapıyorum. Sana İKİNCİ BİR ŞANS veriyorum!"* ve bir sonraki sayfaya geçer.
- Eğer **0 ise** (yani yelkovan tam bir tur atana kadar o sayfaya hiç kimse dokunmamışsa), *"İkinci şansını kaybettin!"* der ve o sayfayı acımadan evden atar.

RAM Savaşları: Çırpınma (Thrashing) ve Çalışma Seti (Working Set)

Bu bölüm, sistemin çaresiz kalıp kilitlendiği o meşhur "donma" anlarının arka planını anlatıyor.

Çerçeve Paylaşımı (Allocation of Frames)

RAM'i devasa bir pizza, açık olan programları da masadaki arkadaşların olarak düşün. İşletim sistemi bu pizzayı (Çerçeveleri/Frames) programlara nasıl dağıtacak?

- **Yerel (Local) Değiştirme:** Her programa "Al sana 50 dilim, ne yaparsan yap" dersin. Bir program kendi hakkını doldurduğunda, sadece kendi içindeki sayfaları diske atıp yenisini alabilir. Güvenlidir, kimse kimsenin dilimini çalamaz ama bir program 50 dilimi bitiremezse o RAM boşuna ziyan olur.
- **Küresel (Global) Değiştirme:** Ortada bir havuz vardır. Yeni bir sayfaya ihtiyacı olan program, gidip *başka bir programın* elindeki çerçeveyi (dilimi) zorla alıp diske atabilir! Acımasızdır ama boş yer kalmamasını sağladığı için (verimli olduğundan) günümüzde en çok kullanılan yöntemdir.

Çırpınma (Thrashing)

Diyelim ki Küresel (Global) sistemi kullanıyoruz. Bilgisayarda çok ağır 10 tane program açtın ve hepsinin çok fazla belleğe ihtiyacı var. Ne olur?

- A programı çalışmak için B'nin sayfasını diske atar. B çalışmak için uyanır, kendi sayfasının olmadığını görür ve gidip C'nin sayfasını diske atarak kendi sayfasını geri getirir.
- Sistem sürekli diskten RAM'e, RAM'den diske sayfa kopyalamaya başlar.
- İşletim sisteminin asıl işi bırakıp sadece sayfaları takas ederek zaman öldürmesine **Çırpınma (Thrashing)** diyoruz.

Çırpınma başladığında çok sinsi bir olay döngüsü gerçekleşir.

1. Bütün programlar diskten veri beklediği için işlemci (CPU) boşa kalır.
2. İşletim sistemi bakar ki CPU kullanımı çok düşmüş. *"Aa işlemci tembellik yapıyor, demek ki sistemi yeterince zorlamıyorum, hemen yeni bir program daha açayım (Multiprogramming derecesini artırayım)!"* der.
3. Yeni açılan program da kendine RAM isteyeceği için var olan o küçücük RAM daha da sıkışır, takas (swap) trafiği iyice felç olur.
4. İşlemci kullanımı **sıfıra (0)** çakılır ve bilgisayar tamamen kilitletlenir.

Kurtuluş Planı 1: Çalışma Seti (Working-Set) Modeli

İşletim sistemi bu felaketi önlemek için "Bölgesellik" (Locality) kuralını kullanır. Bir programın her an tüm sayfalarına ihtiyacı yoktur; sadece o saniye işlem yaptığı "aktif" sayfalarına ihtiyacı vardır. Buna **Çalışma Seti (Working Set)** denir.

- İşletim sistemi bakar: Bütün programların "aktif çalışma setlerinin" toplamı, bilgisayardaki fiziksel RAM'den büyükse, Thrashing (Çırpınma) başlamak üzeredir.
- Programlardan birini (veya birkaçını) **tamamen dondurur ve diske atar** (Suspend/Swap out). Kalanlara rahat bir nefes aldırır, işleri bitince dondurduğu programı geri çağırır.

Kurtuluş Planı 2: Sayfa Hatası Sıklığı (Page-Fault Frequency - PFF)

Çalışma Seti (Working Set) modelini hesaplamak çok zordur. Onun yerine;

- İşletim sistemi bir alt sınır, bir de üst sınır belirler.
- Eğer bir program çok sık "Sayfa Hatası" veriyorsa (üst sınırı aşmışsa), belli ki bu program RAM'de boğuluyor, nefes alamıyor demektir. Ona **yeni boş çerçeveler** verilir.
- Eğer bir program neredeyse hiç "Sayfa Hatası" vermiyorsa (alt sınırın altına düşmüşse), belli ki ona gereğinden fazla RAM verilmiştir, keyfi çok yerindedir. Elindeki fazla çerçeveler **zorla geri alınır**.

İşletim Sisteminin Kendi Hafızasını Yönetmesi

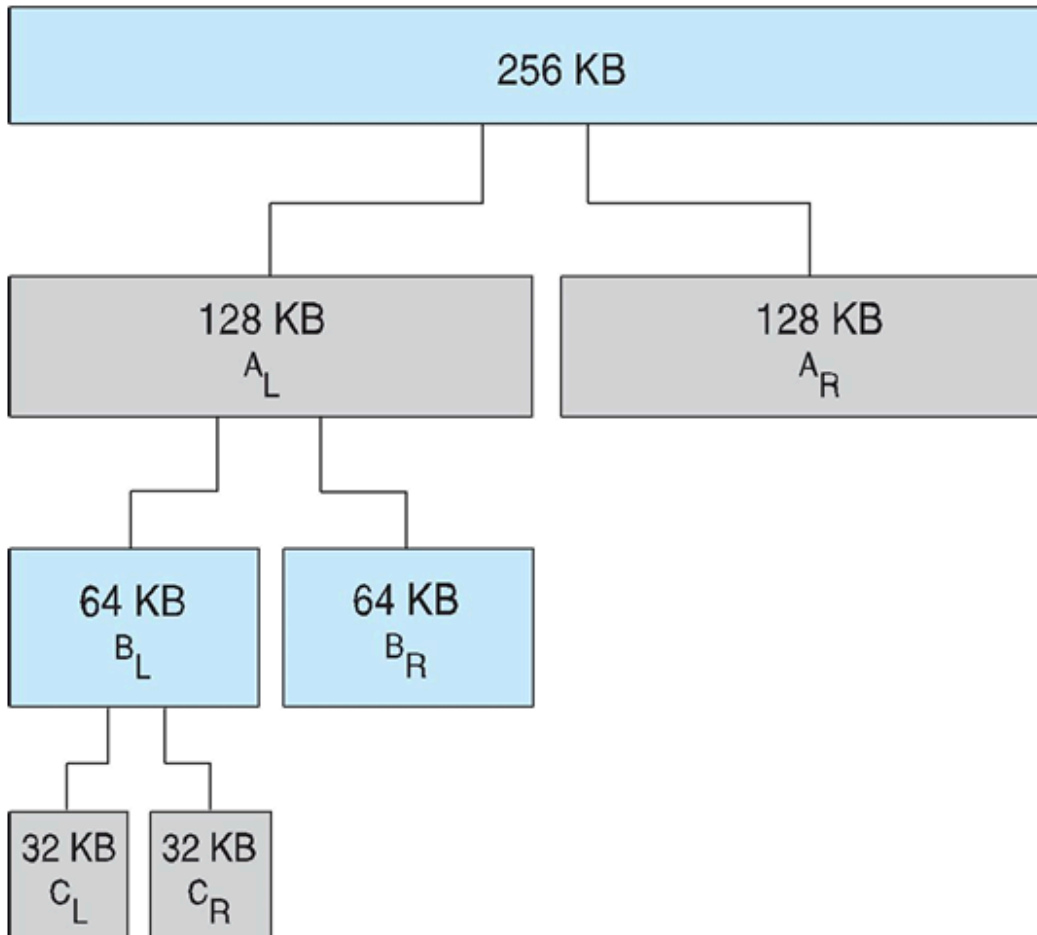
Şimdiye kadar hep "Kullanıcı" programlarının (Chrome, Word, Oyunlar) hafızasını yönettik. Ancak işletim sisteminin kalbi olan **Çekirdek (Kernel)** de bir programdır ve onun da RAM'e ihtiyacı vardır.

Üstelik Çekirdek çok kaprislidir: Kullanıcı programları gibi RAM'in her yerine dağılmış, parça pinçik sayfaları (Paging) sevmez. Özellikle donanımlarla (Örn: USB, Ekran Kartı) haberleşirken hafızanın **fiziksel olarak yan yana (bitişik)** olmasını şart koşar. İşte bu yüzden çekirdeğe özel iki farklı hafıza yönetim sistemi icat edilmiştir.

1. Buddy System (İkizler/Kankalar Sistemi)

Bu sistem hafızayı her zaman **2'nin üsleri** (4, 8, 16, 32, 64...) şeklinde böler.

physically contiguous pages



- Diyelim ki elimizde bütün halinde 256 KB boş bir RAM alanı var ve çekirdek bizden "Bana 21 KB yer lazım" dedi.
- Sistem 256'yı ortadan ikiye böler: 128 KB ve 128 KB (Bunlar birbirinin Buddy'si/ ikizidir). 128 bize hala çok büyük.
- 128'i ikiye böler: 64 KB ve 64 KB. Hala büyük.
- 64'ü ikiye böler: 32 KB ve 32 KB.
- İşte şimdi 21 KB'lık isteği bu 32 KB'lık kutunun içine koyar ve işlemi bitirir.
- **Avantajı:** İş biten kutular yanındaki "ikizi" ile saniyeler içinde birleşip tekrar büyük bir kutu oluşturur. Çok hızlıdır.

- **Dezavantajı (İç Parçalanma):** Çekirdek 21 KB istedi, biz ona mecburen 32 KB verdik. İçerideki 11 KB tamamen çöpe gitti! Bu israfa **İç Parçalanma (Internal Fragmentation)** diyoruz.

Slab Allocator (Kalıp/Önbellek Sistemi)

Buddy sistemindeki o 11 KB'lık israflar birikip sistemi yormaya başlayınca, devrim niteliğinde bir şey buldular.

Çekirdek aslında RAM'den rastgele boyutlarda yer istemez. Sürekli yarattığı ve sildiği belli başlı nesneler vardır.

- Mühendisler der ki: *"Neden her seferinde 2'nin üsleriyle bölüp israf edelim? Sık kullanılan bu nesneler için RAM'de özel kalıplar (Slab'ler) oluşturalım!"*
- Bir Slab üç durumda olabilir: **Dolu (Full)**, **Boş (Empty)** veya **Kısmi (Partial - yarısı dolu yarısı boş)**.
- Yeni bir nesne lazımsa, hemen "Kısmi" slab'den bir yuva verilir. Hiçbir hesaplama yapılmaz. **İç Parçalanma (israf) SIFIRDIR.** (Linux bu yöntemi kullanır; büyük sunucular için SLUB, kısıtlı cihazlar için SLOB varyasyonları vardır).

Sınavların Klasik Mantık Sorusu: Kod Yazım Tarzı

- C dili tabloları RAM'e **"Satır Satır" (Row-oriented)** kaydeder. Yani 1. satırdaki tüm elemanlar aynı 4 KB'lık sayfanın (Page) içindedir.
- **Kötü Programcı:** Döngüyü yazarken tabloyu *Sütun Sütun* okutur. Önce (1,1), sonra (2,1), sonra (3,1)... İşlemci her adımda bambaşka bir satıra (yani RAM'deki bambaşka bir sayfaya) zıplar. Sadece 128×128'lik minicik bir matriste bile **16.384 tane Sayfa Hatası (Page Fault)** yaşanır! Bilgisayar felç olur.
- **İyi Programcı:** Döngüyü *Satır Satır* okutur. İşlemci bir sayfayı RAM'e getirir, içindeki 128 elemanı da okur, sonra diğer sayfaya geçer. Toplam sadece **128 Sayfa Hatası** ile kod saniyeler içinde biter. Bir kodun donanımı ve OS'i bilerek yazılması işte bu kadar kritiktir!

Performans İnce Ayarları

- **Önceden Sayfalama (Prepaging):** Program açılırken "Birazdan bunlar da lazım olur" deyip diskteki bazı sayfaları RAM'e peşinen getirmektir. Tutar da kullanılırlarsa hız kazandırır, kullanılmazlarsa RAM'i ve diski boş yere yorar.
- **TLB Reach (TLB Kapsamı):** Sayfa Tablolarını anlattığımız PDF'teki o sihirli, süper hızlı donanımsal VIP Not Defterini (TLB) hatırla. Eğer 4 KB'lık sayfalar kullanıyorsan TLB sadece birkaç MB'lık adresi hafızasında tutabilir. Ama donanım (işlemci) sana **Çoklu Sayfa Boyutu** (Örn: 2 MB veya 1 GB'lık dev sayfalar) desteği verirse, o küçücük TLB bir anda devasa bir haritayı kapsayabilir. Modern işlemcilerin sırrı budur.
- **I/O Kilidi (I/O Interlock / Pinning):** Bir program diskten bir dosya kopyalarken (Örn: USB'ye dosya atarken), o verinin tutulduğu sayfa RAM'e kilitlenir (Pinned). Aksi takdirde Sayfa Değiştirme (Page Replacement) algoritması gelip *"Aa bu sayfa boş duruyor"* deyip onu evden atarsa (evict), USB'ye RAM'deki bambaşka, rastgele bir veri (belki de şifrelerin) yazılır!